

Makefile Course

Mục tiêu khóa học

- Nắm được Build Process cơ bản trong hệ thống nhúng
- Biết cách sử dụng Compiler GCC
- Nắm bắt được các khái niệm cơ bản về Makefile
- Cách viết Makefile cho một project Vi điều khiển
- Tổng quát hóa Makefile cho các project khác nhau

Giới thiệu

- Tác giả: Nguyễn Văn Nghĩa
 - SĐT/Zalo: 0986.951.957
 - Facebook:
 - Cá nhân: <https://www.facebook.com/nghia12a1>
 - Nhóm: <https://www.facebook.com/groups/laptrinhdientu>
 - Blog: <https://laptrinhcnhung.blogspot.com/>
- Tham khảo:
 - Tài liệu Makefile GNU
- Slide giữ các từ khóa/mô tả bằng tiếng anh

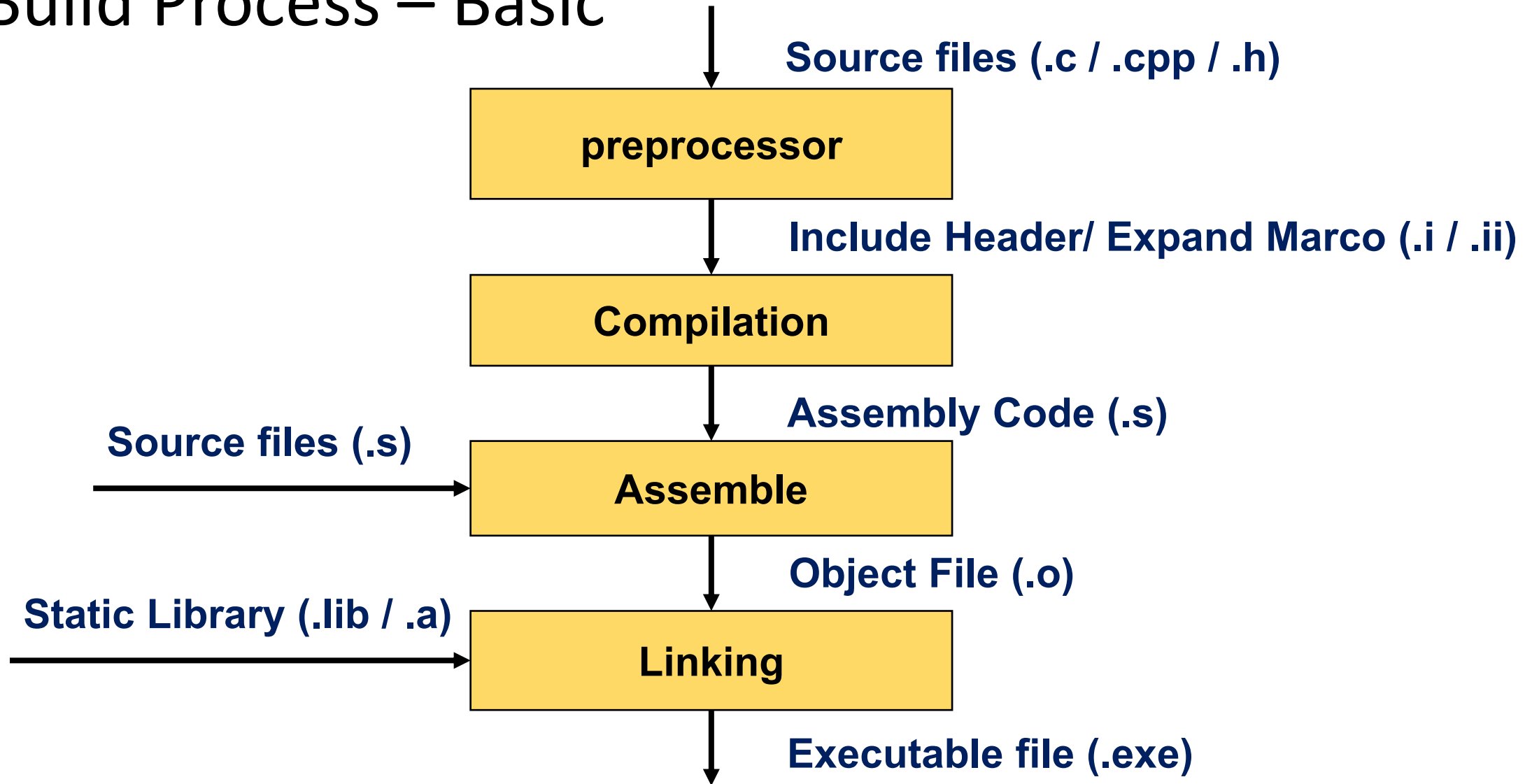
Nội dung

- 1 Build Process
- 2 GCC – Command Line
- 3 Chương trình Makefile đơn giản
- 4 Makefile Vi điều khiển
- 5 Tổng quát hóa Makefile

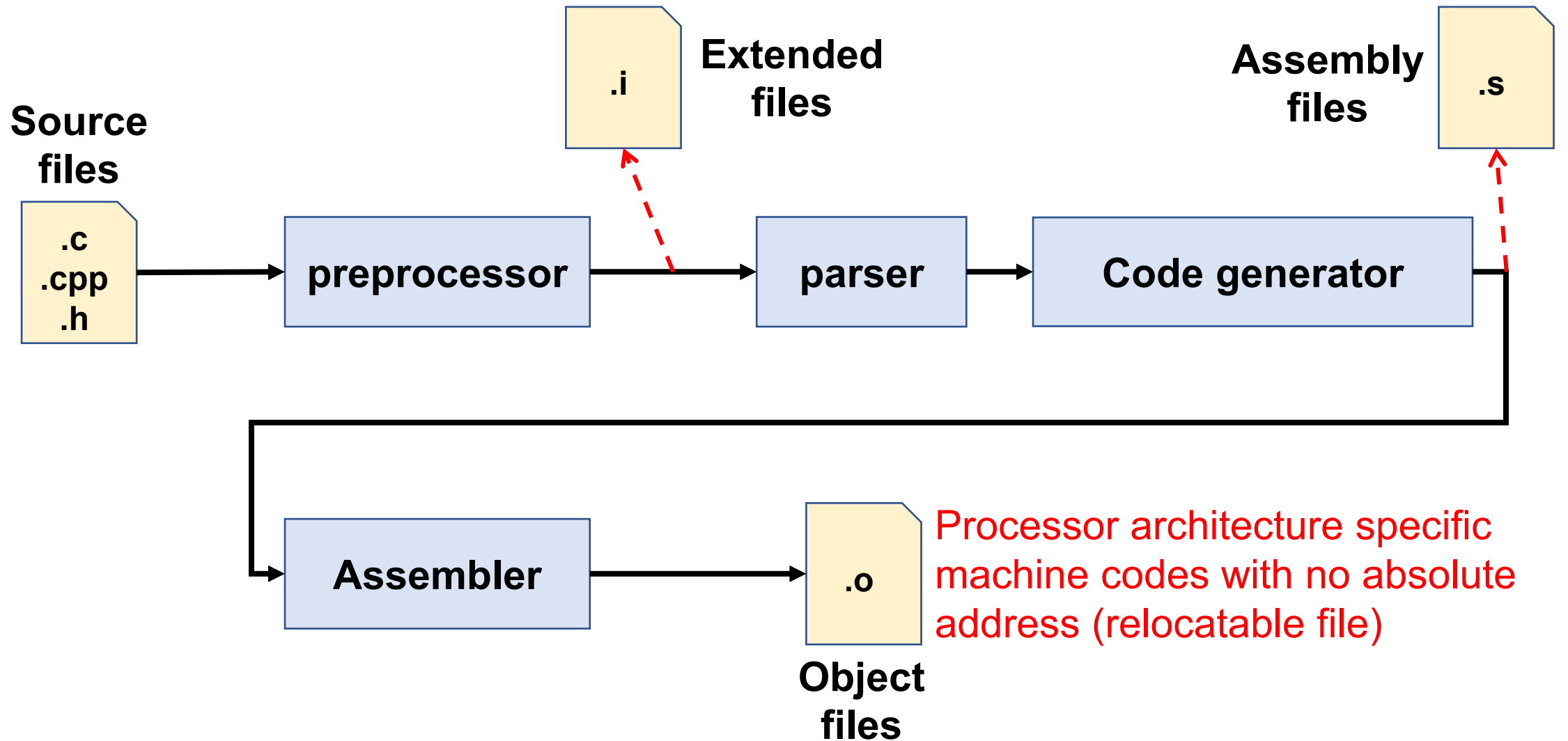
Build Process

Từ Source Code đến file nhị phân nạp vào Target

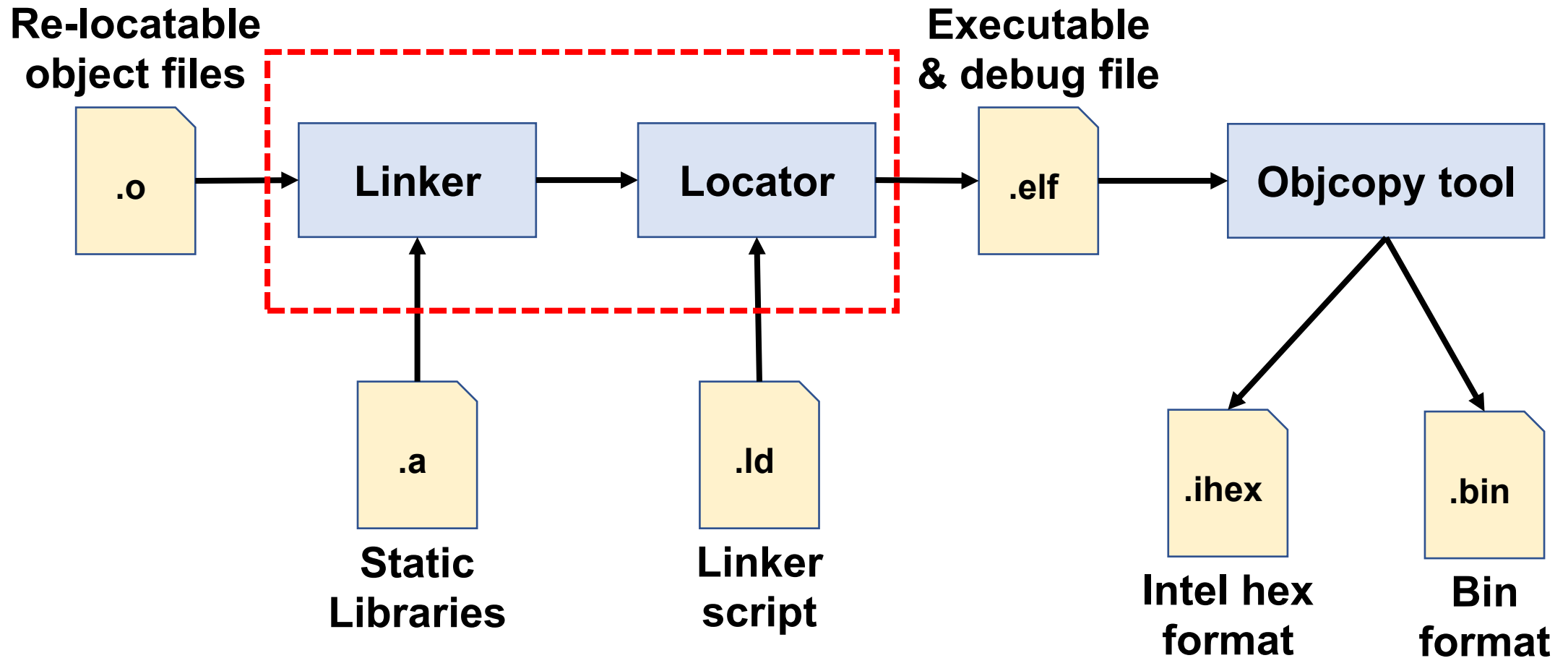
Build Process – Basic



Build Process – Embedded



Build Process – Embedded



Đọc thêm

- Preprocessor
<https://laptrinhcnhung.blogspot.com/2021/08/Preprocessor.html>
- Build Process
<https://laptrinhcnhung.blogspot.com/2021/08/Build.html>
- Cross Compiler
<https://laptrinhcnhung.blogspot.com/2021/08/CrossCompiler.html>
- Quá trình Linking
<https://laptrinhcnhung.blogspot.com/2021/08/Linking.html>
- Static Library vs Shared Library
<https://laptrinhcnhung.blogspot.com/2021/08/Library.html>
- Linker Script File
<https://laptrinhcnhung.blogspot.com/2021/08/LinkerScriptFile.html>

GCC – Command Line

Giới thiệu trình biên dịch GCC

Command Line cơ bản và sử dụng GCC với Build Process

Makefile - Lí do cần sử dụng Makefile

Trình biên dịch là gì?

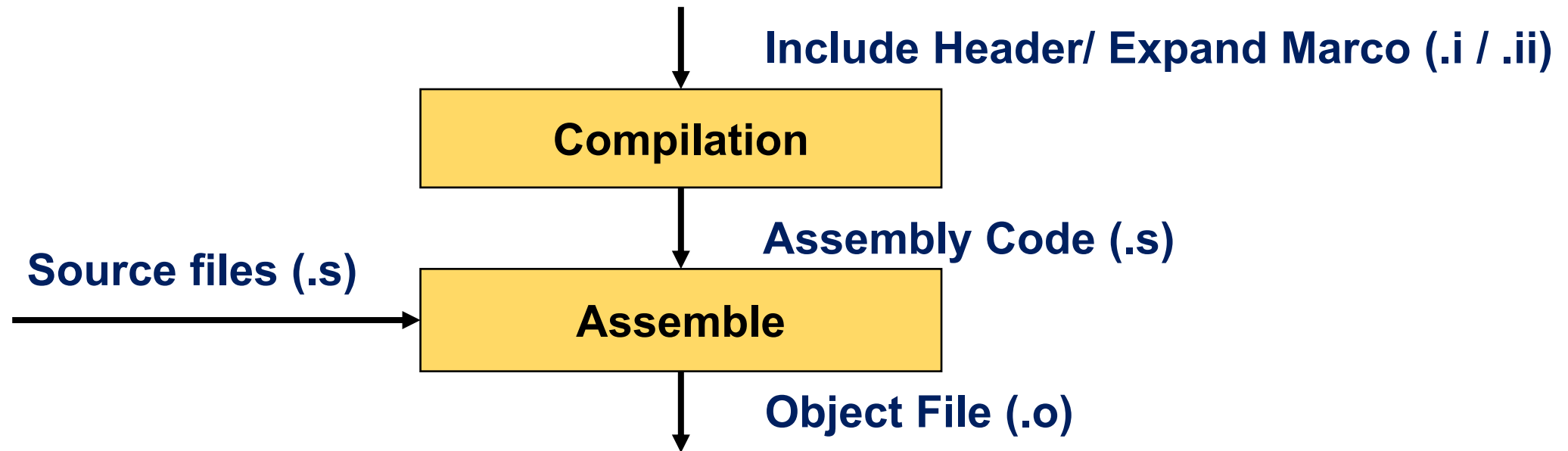
- “*Compiler (trình biên dịch) là một chương trình máy tính làm công việc dịch các chuỗi câu lệnh viết bằng ngôn ngữ lập trình thành chương trình tương đương nhưng viết dưới dạng ngôn ngữ máy tính. Chương trình mới này được gọi là mã đối tượng (object code)*”.



- Generate ROM-able code
- Generate Optimized code
- Generate Re-entrant code
- Support for Different Members in Microcontroller Family
- Support for Different Memory Models

GCC là gì?

- GCC - GNU Compiler Collection - tập hợp các trình biên dịch được thiết kế cho nhiều ngôn ngữ lập trình khác nhau
- Thành phần quan trọng của GNU toolchain



Cài đặt GCC

- GCC có sẵn khi cài CodeBlock/Dev C++

- GCC – MingW

<https://codecute.com/c/huong-dan-cai-dat-trinh-bien-dich-c-c-mingw-gcc.html>

- GCC – Cygwin

<https://codecute.com/c/cygwin-la-gi-cai-dat-compiler-c-c-cygwin-tren-windows.html>

```
E:\Video_MCU\C_Video\GCC\GCC>gcc --version
gcc (GCC) 10.2.0
Copyright (c) 2020 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Command Line

- Kiểm tra GCC version
- Tạo folder mới
- Tạo một file source
- Tạo một file mới

```
gcc --version
```

```
mkdir <folder_name>
```

```
cd <folder_son>
```

```
touch <file_names>
```

Command Line

- Preprocessor
- Compilation
- Assemble
- Linking

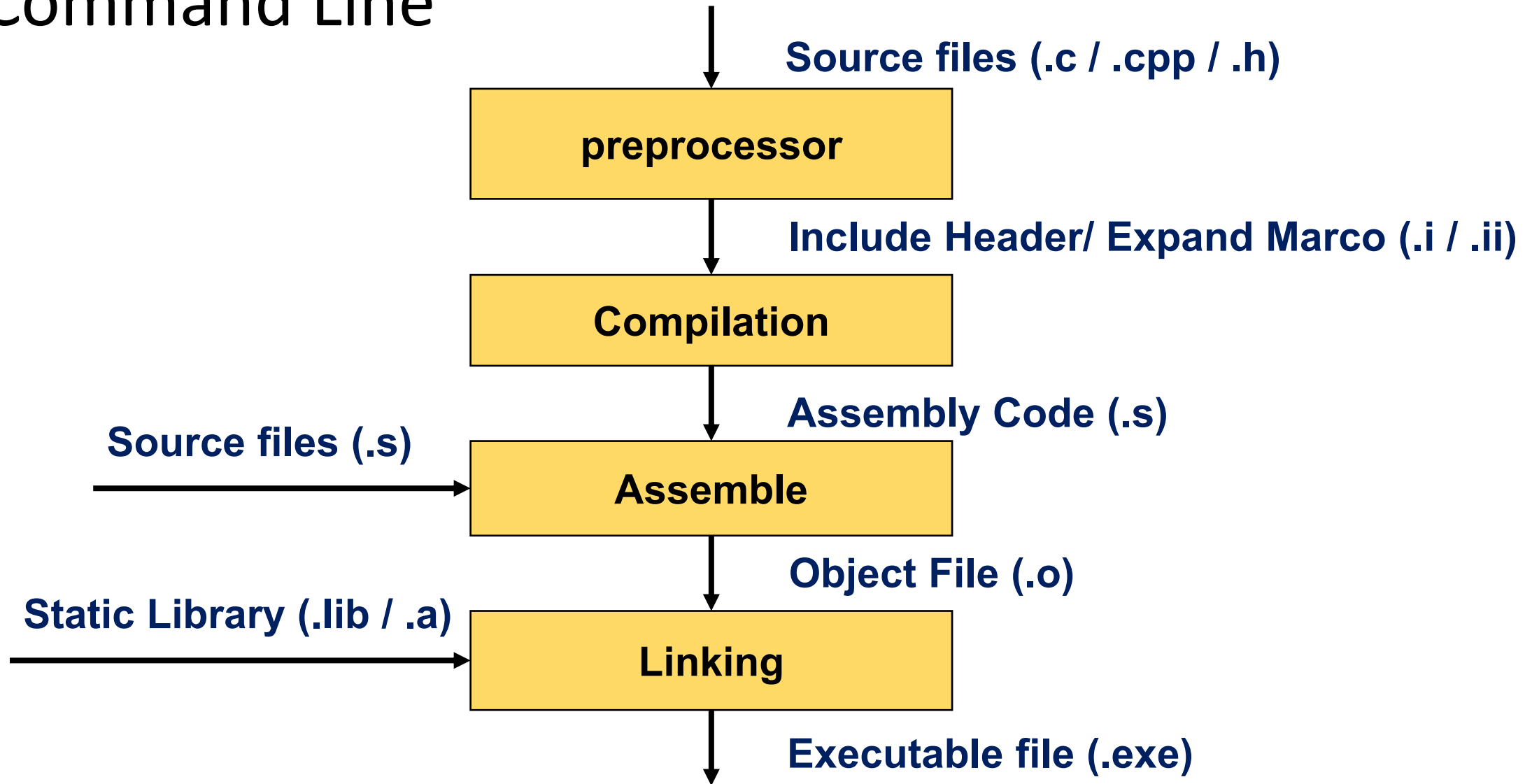
```
gcc -E main.c -o main.i
```

```
gcc -S main.i -o main.s
```

```
as main.s -o main.o
```

```
gcc -v -o main main.o
```

Command Line



Chương trình Makefile đơn giản

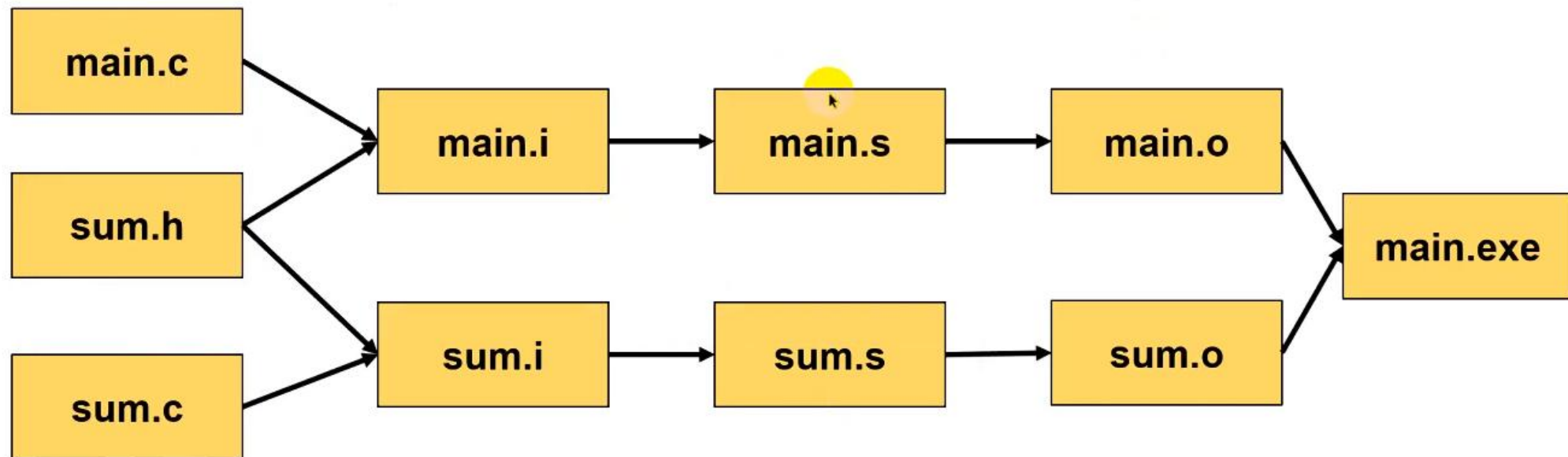
Tại sao cần sử dụng Makefile

Khái niệm cơ bản trong Makefile

Viết Chương trình Makefile đơn giản với C

Bài toán

- main.c / sum.c / sum.h



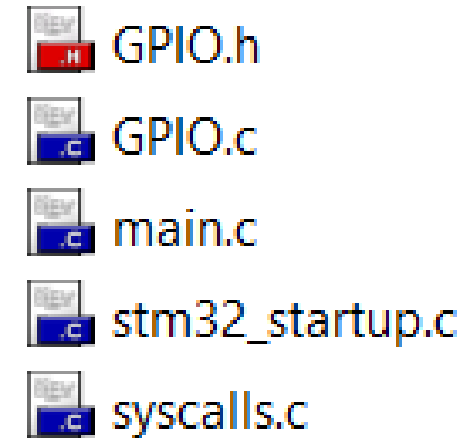
Why you need makefile?

Your project have many source files and you build it by an IDE (Keil C)

At a time, you want to edit or add a file



Add this file to the project
& **Rebuild all file**



Makefile???

You **just need to build**
edited/added files: led .c

Build only



It **spend many time** to
rebuild with a big project

What is Makefile?

- Makefile is a script file format that contains some information:
 - Project structure (file, dependencies)
 - Commands to create files (object file, elf/bin/hex files, ...)
 - The make command will read makefile content, understand project's structure and execute the commands
- Tools for Makefile:
 - Cygwin/Mingw/GNU Make

```
C:\Users\Admin>make --version
GNU Make 3.81
Copyright (C) 2006 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.
There is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A
PARTICULAR PURPOSE.

This program built for i386-pc-mingw32
```

From GCC commands to Makefile

My project:

main.c → **main.o** → **main.exe**

With GCC:

```
gcc -c main.c -o main.o
```

Generate object file

```
gcc main.o -o main.exe
```

Generate .exe file

```
main.exe
```

Run .exe file

With Makefile:

```
main.o: main.c
    gcc -c main.c -o main.o
build: main.o
    gcc main.o -o abc.exe
run:
    abc.exe
```

Combine GCC commands
into one script file

main.c

```
#include <stdio.h>

int main(void)
{
    printf("Hello\n");
    return 0;
}
```

Results

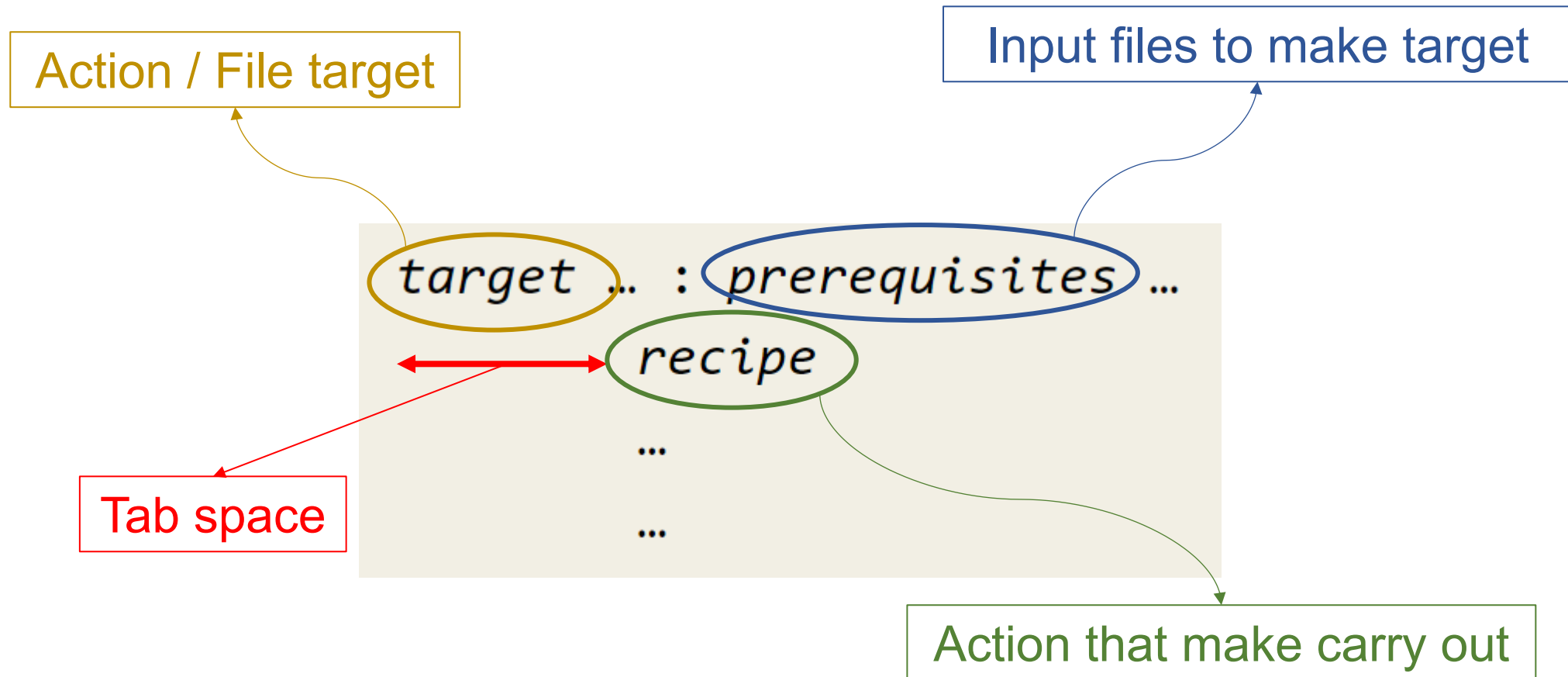
Hello

Cài đặt Cygwin

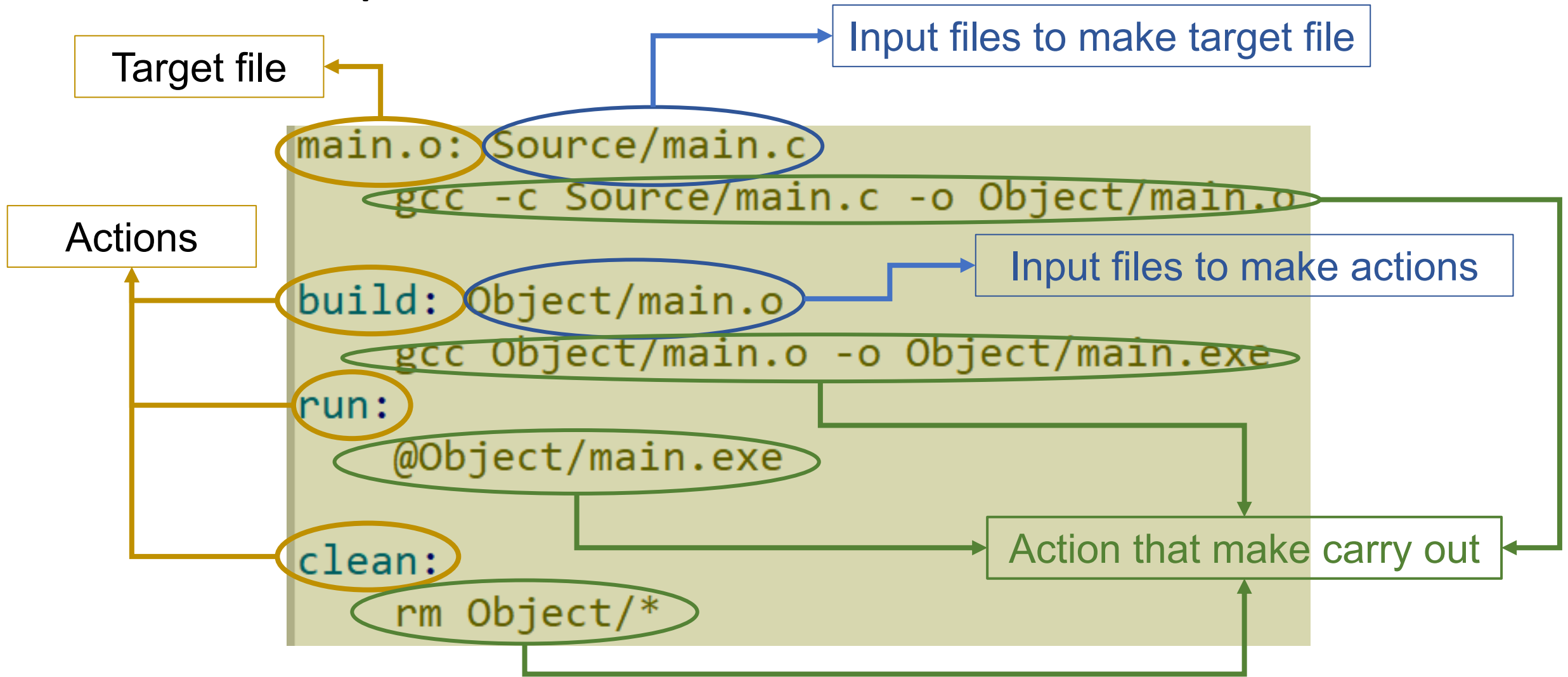
- <https://www.cygwin.com/>
Đã cài đặt ở bài số 2 – Lưu ý, cài đặt package **GCC** và **make**

Rules Makefile

- Basic form of a basic simple makefile



For examples

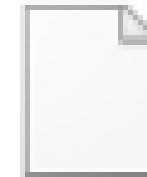


Một số khái niệm cơ bản trong Makefile

PHONY keyword

- I have a file has the same name with an action

```
build: main.o  
    gcc Object/main.o -o Object/main.exe  
run:  
    @Object/main.exe
```



build

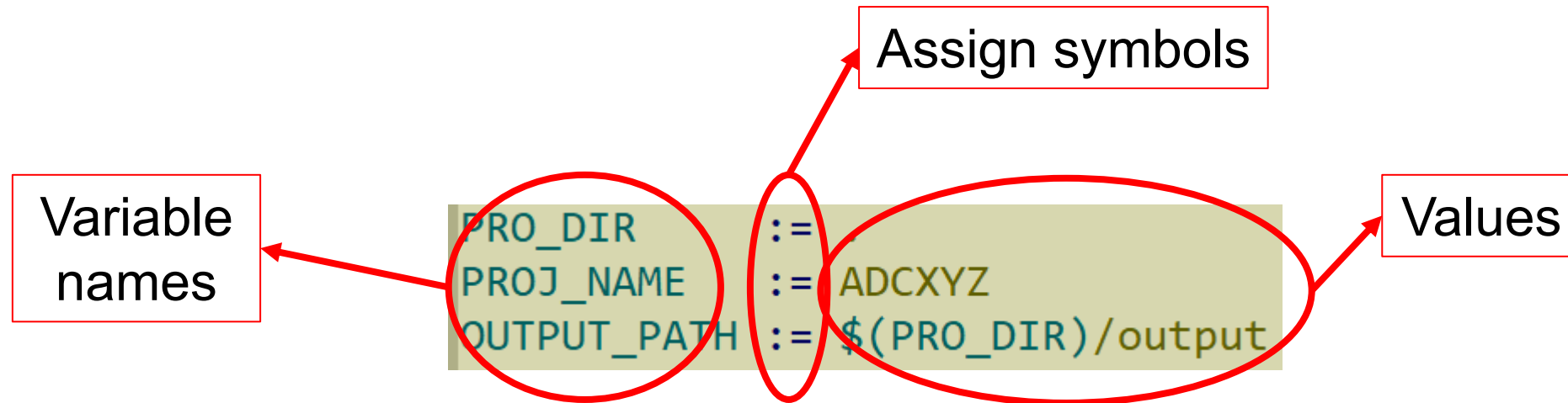
→ Action 'build' is not running with make command

↓
Add PHONY keyword to signal 'build' is a action

```
.PHONY: build  
build: main.o  
    gcc Object/main.o -o Object/main.exe
```

Variables in Makefile

Use to generalize makefile, avoid using specific names of files



Symbol to access the value of a variable: **`$(variable_name)`**

A diagram illustrating variable access in a Makefile. A code block contains two lines: `PROJ_NAME := ADCXYZ` and `$(PROJ_NAME)`. A red arrow points from the variable name `PROJ_NAME` in the first line to the variable access symbol `$(PROJ_NAME)` in the second line.

Print value of variables

- Printing value of variables to check, debugger when working with makefile → Use **'echo'** command & string substitution

```
print-%:  
    @echo $($subst print-,,$@))
```

Results

```
$ make print-PROJ_NAME  
ADCXYZ
```

Tài liệu makefile

- Tài liệu

<https://www.gnu.org/software/make/manual/make.html>

Chương trình Makefile - Vi điều khiển

Makefile với Vi điều khiển

Thực hành viết Makefile với Chương trình Vi điều khiển

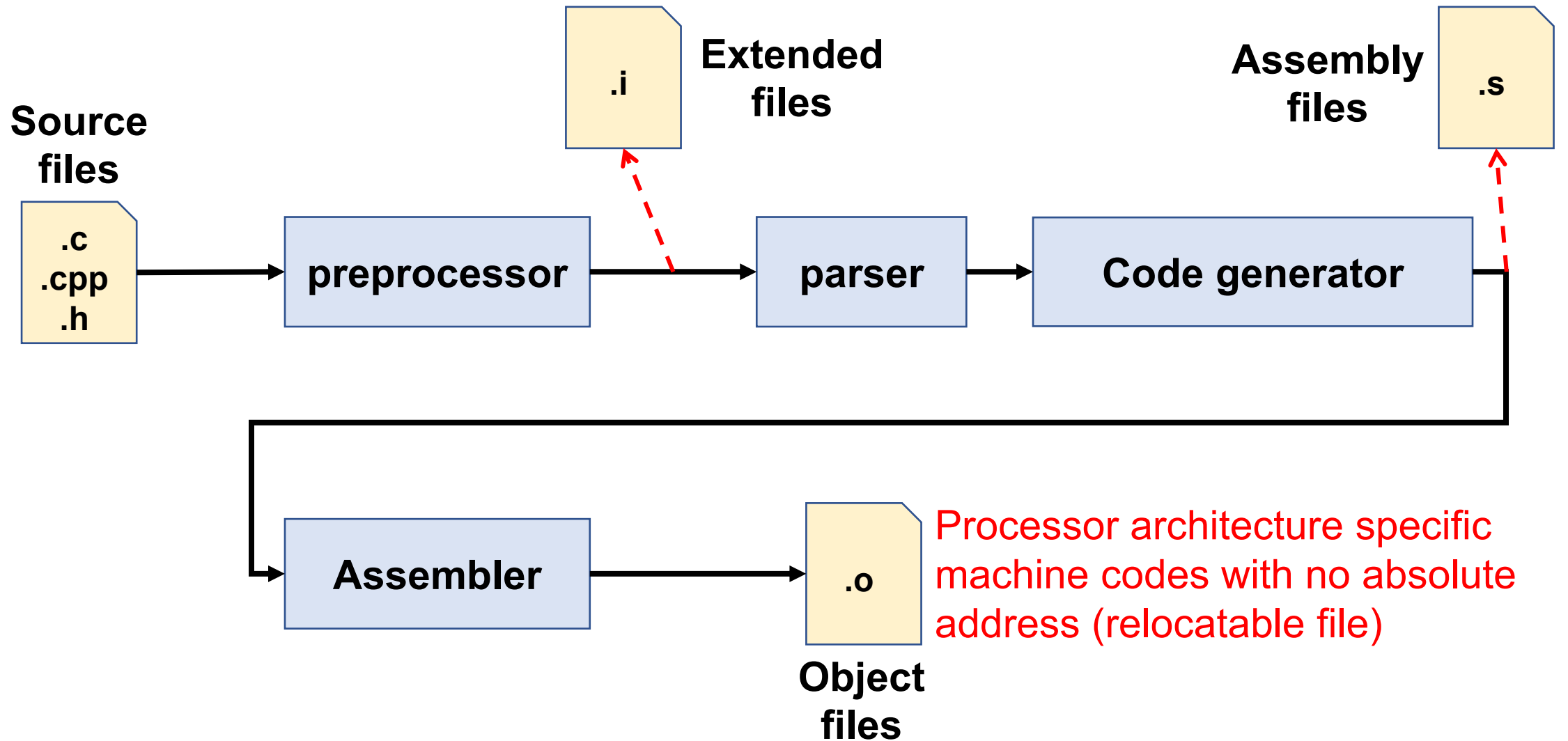
Cài đặt Phần mềm

- Toolchain – GCC for ARM
<https://developer.arm.com/tools-and-software/open-source-software/developer-tools/gnu-toolchain/gnu-rm/downloads>
- Phần mềm nạp code STM32 ST-LINK Utility
<https://www.st.com/en/development-tools/stsw-link004.html>
- Tài liệu
<https://www.gnu.org/software/make/manual/make.html>
- Source Code
<https://github.com/nghia12a1-t-ara/makefile>

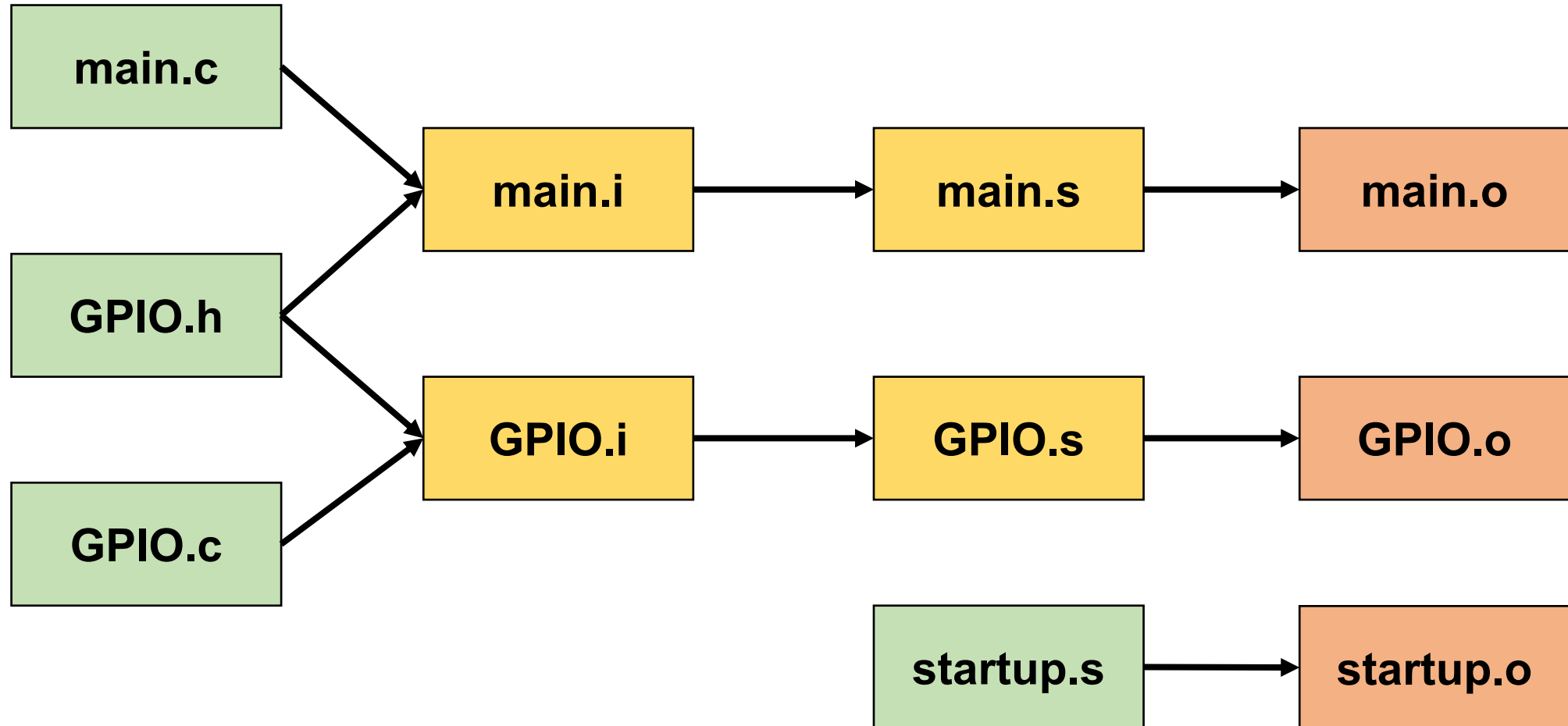
Bài toán

- Viết Makefile cho Chương trình Vi điều khiển sau:
 - Blink LED GPIO
 - Source Code (GPIO.c / GPIO.h / main.c)
 - Startup File, Linker File – Được cung cấp
 - Phần mềm hỗ trợ nạp code
 - STM32 ST-LINK Utility
 - Phần cứng: STM32 (Mình sử dụng STM32F401 – NUCLEO Board)

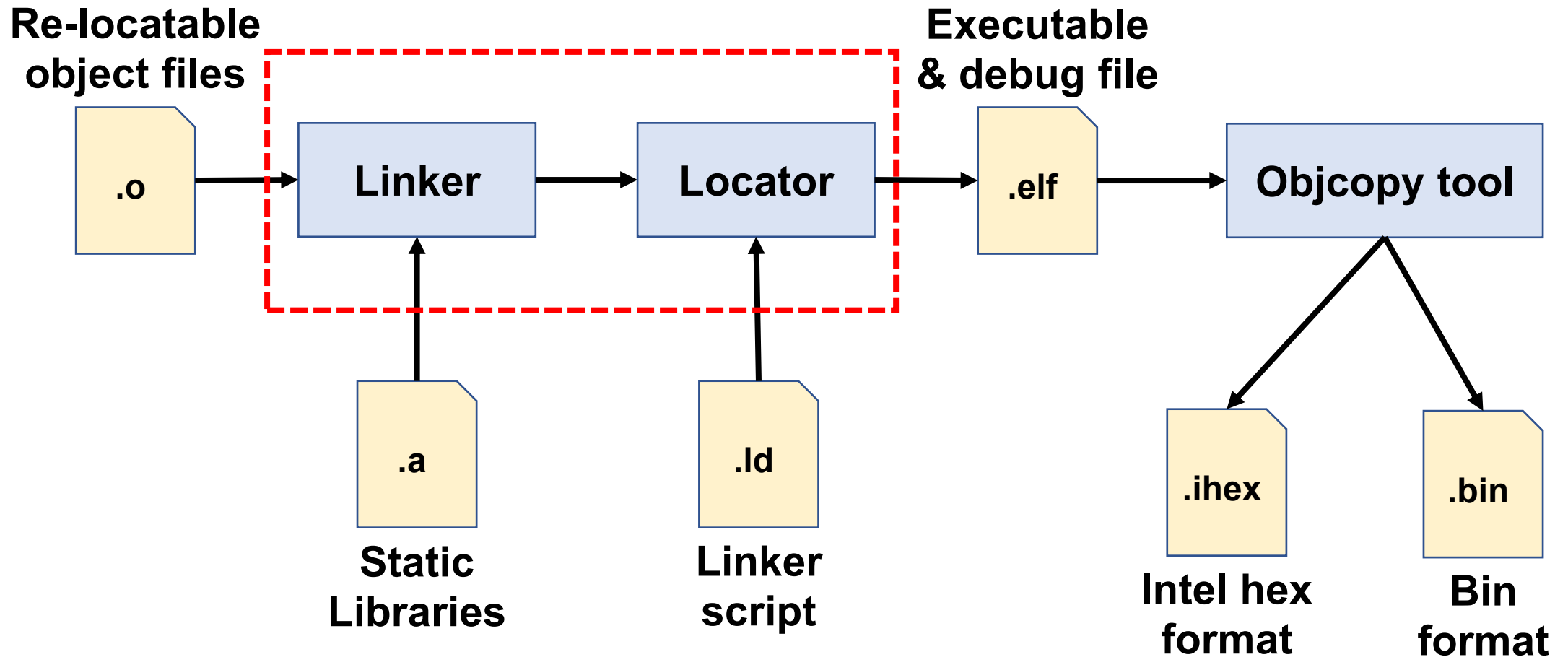
Build Process



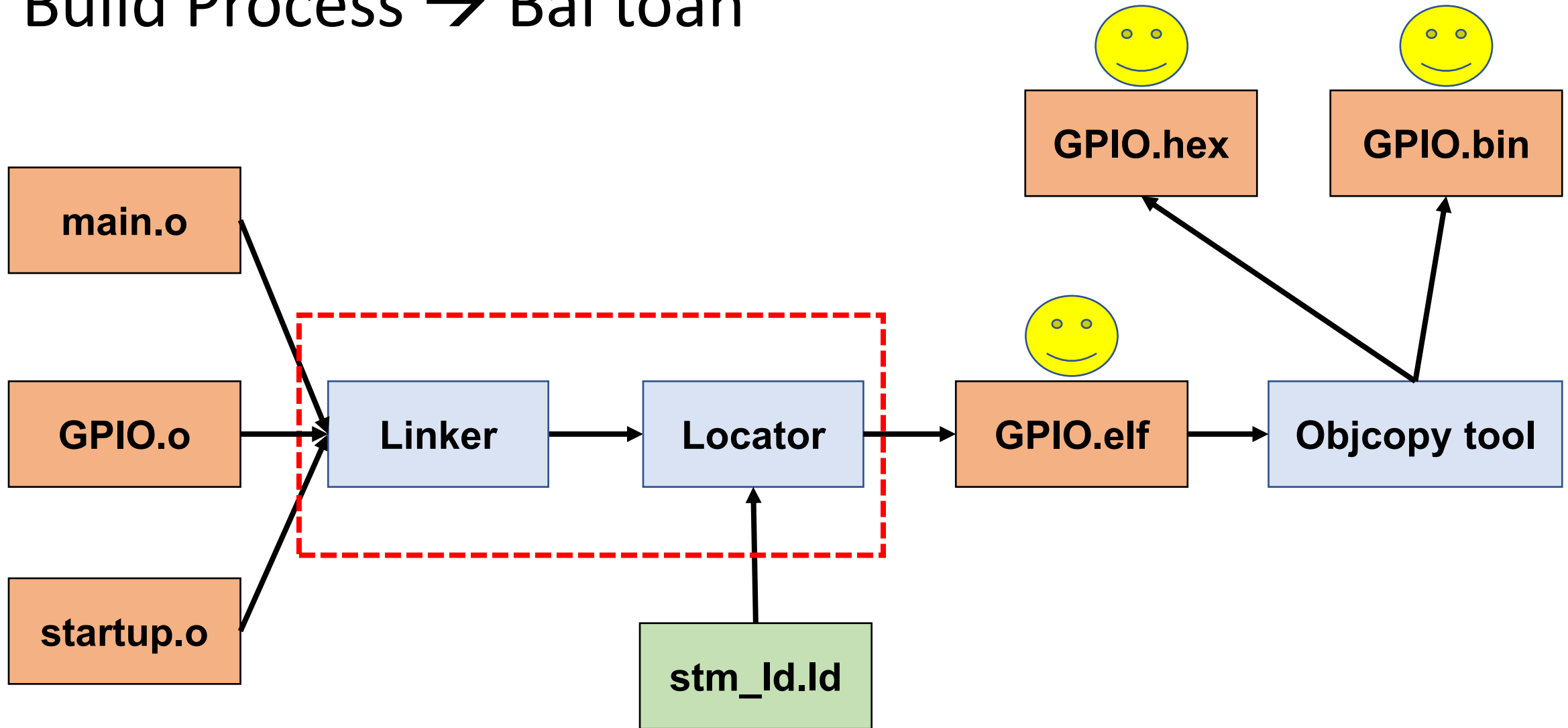
Build Process → Bài toán



Build Process



Build Process → Bài toán



Command

- Compile / Linker

<Compiler Options> -o <Target> <All Prerequisite>

< Linker Options> -o <Target> <All Prerequisite>

- Create **hex, bin, elf** for Microcontrollers

Linker → file elf

< Linker Options> -o <Target> <All Prerequisite>

Object Copy to Create hex file

objcopy.exe -O ihex <file elf> <file hex>

- Nạp code

ST-LINK_CLI.exe -ME

ST-LINK_CLI.exe -p <file hex> <Địa chỉ bắt đầu nạp code>

ST-LINK_CLI.exe -rst

Compiler Options

- **arm-none-eabi-gcc**
- **-mcpu=cortex-m4**
- **-mthumb**
- **-std=gnu11**
- **-Ox**
- **-I\$(Include_DIR)**

Linker Options

- **arm-none-eabi-ld**
- **-T \$(LINKER_FILE)**
- **-Map <path_to_file.map>**
-Wl,-Map= <path_to_file.map>
- **-nostdlib**

Tổng quát hóa Makefile

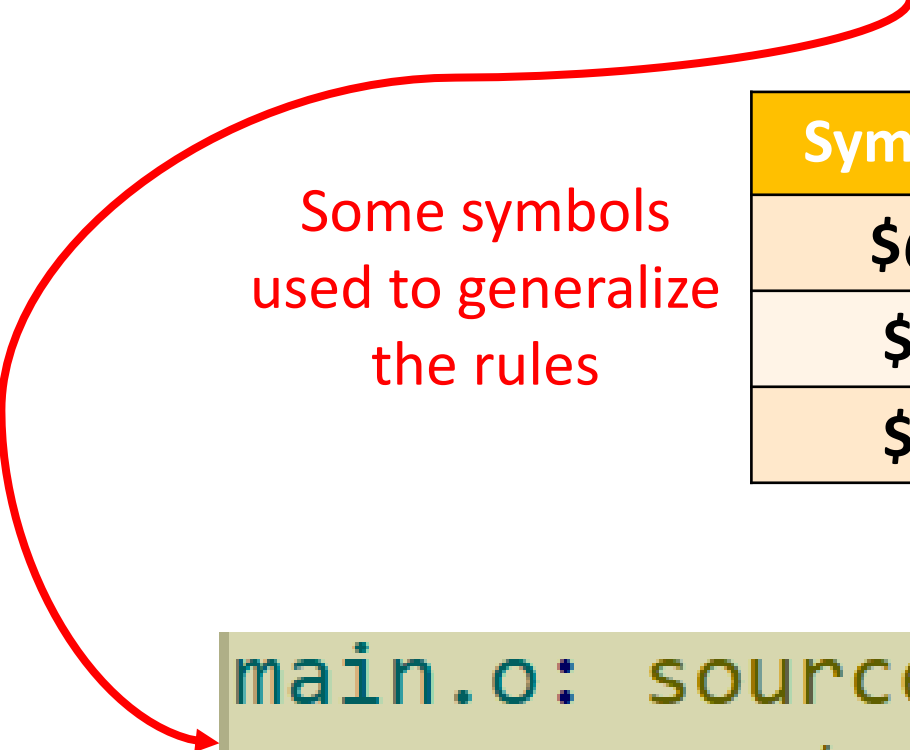
Các từ khóa/function trong makefile

Thực hành tổng quát hóa Makefile với bài toán VĐK

Automatic Variables

```
main.o: source/main.c
    gcc -c source/main.c -o output/main.o
```

Some symbols
used to generalize
the rules



Symbols	Meaning
<code>\$@</code>	File name of the target of the rule
<code>\$\$</code>	Name of all the prerequisite
<code>\$<</code>	Name of the first prerequisite

```
main.o: source/main.c
    gcc -c $$ -o output/$@
```


'wildcard' Characters

- The wildcard characters in make are '*', '?', '[...]'
- For examples, *.c specifies a list of all the files whose names end in '.c'

```
clean:
```

```
@rm output/*
```

Clear all files in 'output' folder

```
SRC_DIRS = C:/Users/Admin/OneDrive/Desktop/FPT/  
SRC_FILES := $(wildcard $(SRC_DIRS)/*)
```

Get all files in the PATH
C:\Users\Admin\...

```
$ make print-SRC_FILES  
C:/Users/Admin/OneDrive/Desktop/FPT/GAM_DAP_Training/Makefile/Examples/Helloworld/source/main.c
```

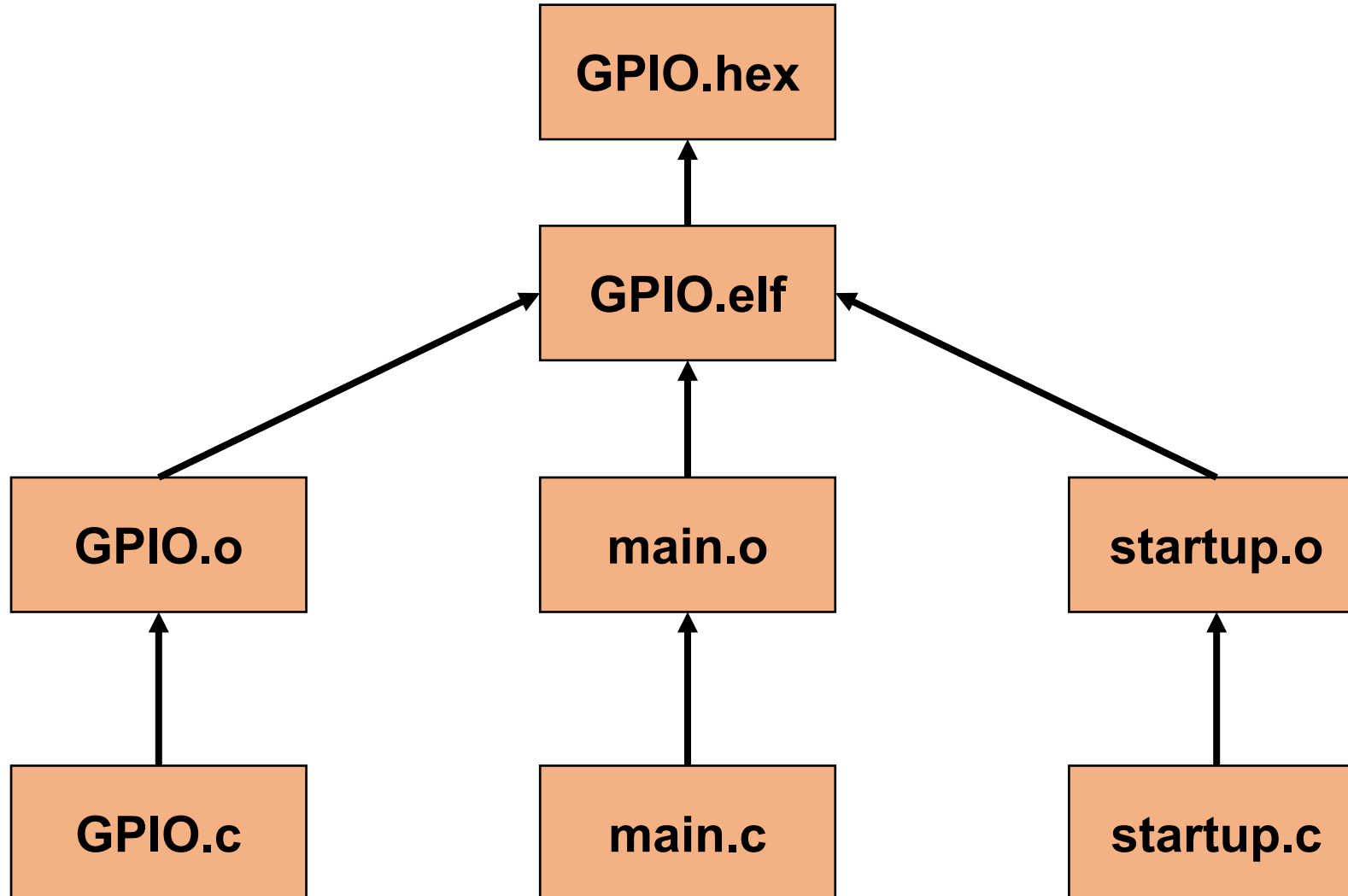
‘foreach’ function

- This function causes one piece of text to be used repeatedly, each time with a different substitution performed on it.

```
dirs := a b c d  
files := $(foreach dir,$(dirs),$(wildcard $(dir)/*))
```

Get all files in the all folder of **dirs**.

Make Process



VPATH

- Your project has many files in the different folders
- VPATH variable specifies the list of directories that make should search for prerequisites and targets of rules
- VPATH automatically search all Directory for Files 😊

```
#search path .c and .h files  
vpath %.c $(SRC_DIRS)  
vpath %.h $(INC_DIRS)
```

Pattern Rules

`%.o : %.c`

recipes....

→ Tell how to make a .o file from a respectively .c file

‘notdir’ – not direct

- Use to get file or folder without the direct/path

```
OJB_FILES := $(notdir $(SRC_FILES))
```



```
$ make print-OJB_FILES  
main.c sub.c sum.c
```

Other Concept in Makefile

Ifeq & export

- Use many module in a test program
→ Modify Makefile with each module??? → NO
- We can use export

```
# Warning with empty module
ifeq ($(MODULE),)
    $(error export MODULE before starting tests)
endif

PRO_DIR := .
PATH_OUTPUT = ./output
PROJ_NAME := $(MODULE)

INC_DIRS = D:/FPT/GAM_DAP_Training/Makefile/Examples/Endian_Test/driver/$(MODULE)/include
```

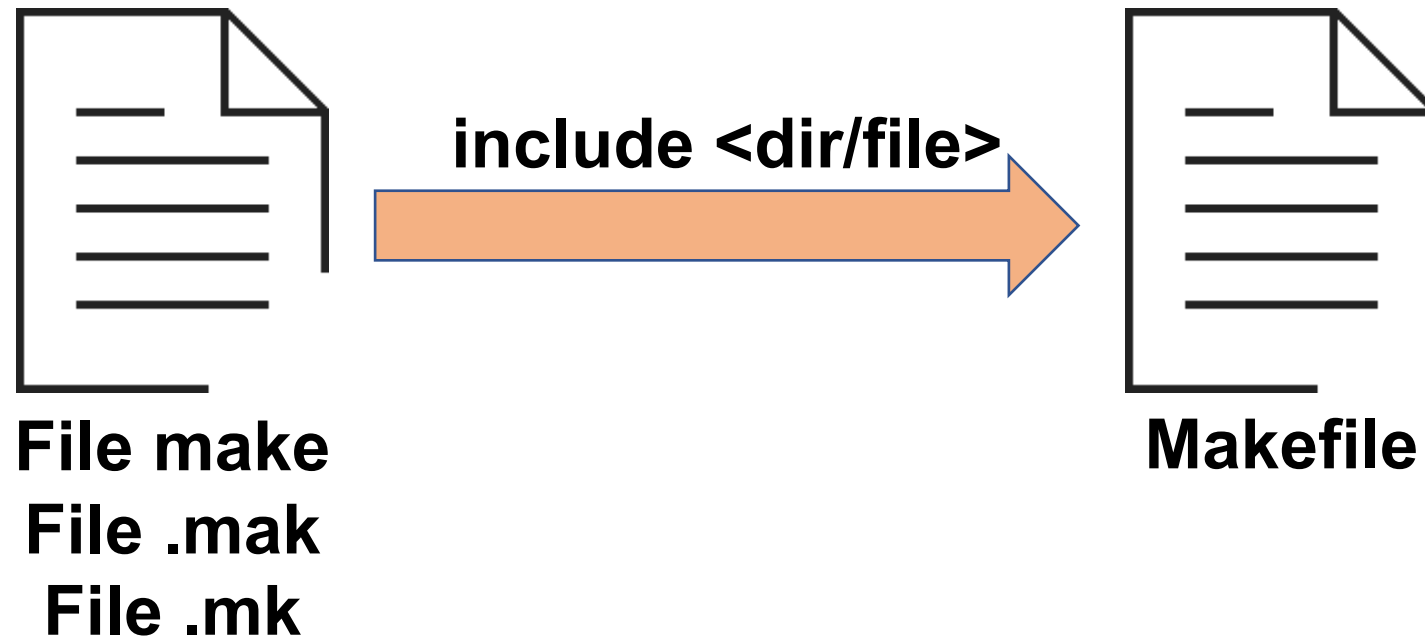
Handwritten red annotations: "gpio" is written next to `$(MODULE)` in the `PROJ_NAME` line. Red circles highlight `$(MODULE)` in the `PROJ_NAME` and `INC_DIRS` lines.

```
Admin@Admin /cygdrive/d/FPT/GAM_DAP_Training/Makefile/Examples/Endian_Test/framework
$ export MODULE=gpio

Admin@Admin /cygdrive/d/FPT/GAM_DAP_Training/Makefile/Examples/Endian_Test/framework
$ make clear; make build
```


Include Makefile

- Include other makefile or file config of makefile



Thank
you!

The image features the words "Thank you!" in a highly decorative, hand-drawn style. The letters are thick and outlined in black. The word "Thank" is positioned above "you!". The letters are filled with various colors: 'T' is yellow, 'h' is orange, 'a' is green with a dotted pattern, 'n' is purple, 'k' is orange, 'y' is green with a dotted pattern, 'o' is pink with a dotted pattern, 'u' is blue with a dotted pattern, and the exclamation mark is blue with a green center. The letters are adorned with various patterns, including zig-zags, dots, and spirals. Several colorful flowers are scattered around the text: a blue flower with a purple center is at the top right, a blue flower with a purple center is to the right of the 'k', a large pink flower with a yellow center is in the bottom left, a blue flower with a purple center is to the left of the 'y', and a small blue flower with a green center is at the bottom right. The background is white, and the entire illustration is set against a dark blue border.